

CENTURY NIT CONSULT

Complete Platform Documentation

The study-abroad platform end to end. What it does, how it is built, how the client journey works, and how it runs in production.

Contents

Part 1	What the platform is	3
Part 2	Signing in	4
Part 3	Who can do what	5
Part 4	The client journey	6
Part 5	Consultations, scheduling and meetings	9
Part 6	Money	10
Part 7	Documents	11
Part 8	Communication	12
Part 9	Marketing	13
Part 10	Content management	15
Part 11	The Operations Center, page by page	16
Part 12	The website and portal, page by page	17
Part 13	Data, security and the background engine	18
Part 14	Under the hood, the flows that matter	20

Everything about the platform in one document, in plain language. It covers what the system is, how it's built, what each part does, how the client journey works end to end, and how it runs in production.

What the platform is

Century NIT is a study-abroad consultancy platform. A client signs up, books a paid consultation, and is then guided step by step, by real staff, through enrolment, university applications, visa processing, and departure. Staff run the whole operation from a separate console.

The product is **three deployable pieces**:

1. **The public website and client portal**, where visitors browse and where clients live their journey.
2. **The Operations Center**, the staff-only console for consultations, cases, finance, marketing, content, and administration.
3. **The API**, the single backend that both front ends talk to. It holds all data and all business rules. It also publishes an interactive reference (every endpoint, its inputs, and a working "try it" button) for anyone integrating against it.

A visitor to the website never downloads a byte of the staff console. They are separate applications with separate builds.

HOW THE PIECES CONNECT

Each front end sits behind its own Cloudflare Worker. The Worker serves the app's files and forwards every API call to the backend. Because of that forwarding, the browser sees one origin per app, the login cookie is first-party, and sign-in just works. The API keeps a strict list of allowed origins for the rare calls that are genuinely cross-origin; the same list feeds both the browser security layer and the sign-in system so they can never drift apart.

WHERE THE TRUTH LIVES

Everything of record is in one PostgreSQL database: cases, invoices, documents, messages, marketing, content. Both front ends are thin clients. They show what the API says and send back what the user does. The portal keeps a little per-browser convenience state (drafts, dismissed hints) and an offline fallback for the journey view, but nothing there is authoritative.

PART 2

Signing in

One identity system serves both audiences. A staff member is simply a user account that also has a staff record attached; that record is where their role and branch live.

CLIENTS

Clients register themselves. They can sign in with:

- Email and password (with email verification and password reset)
- Google
- Phone number with a texted code (only when a text provider is configured; without one the option refuses rather than pretending)
- A one-time code emailed to them (passwordless)

Clients can add two-factor protection, an authenticator app or emailed codes, but it's never required. An applicant is not made to install an authenticator before they can book a consultation.

STAFF

Staff can never register themselves. There is no staff sign-up anywhere. An account exists only because someone senior invited it, and the invitee sets their own password; nobody else ever knows it. Invitations and one-time codes go out by email; in development they print to the server console instead so everything can be tested without email accounts.

Two-factor is **required for every staff role**, enforced by the server: either an authenticator app or enrolled email codes. The very first administrator is created once with a server-side setup token; the moment any staff member exists, that door closes permanently.

Who can do what

Two separate ideas, both checked on the server (the console interface only hides buttons; the backend is the real authority):

- **Modules** decide what a role can *see*: which pages and API areas open to them.
- **Capabilities** decide what a role can *do*: assign work, issue invoices, invite staff, own a chapter of a case.

Both live in one shared definition used by the console and the API, so they can never disagree.

Administrators can build custom roles in a permission matrix; a custom role gets a rank, and you can only grant roles ranked below your own.

THE BUILT-IN ROLES

ROLE	SEES	CAN DO
Super Admin	Everything	Everything
Admin	Platform administration (staff, sign-in policy, content, settings) plus read access to casework	Invite and manage staff, manage roles and client accounts, change platform settings; does not work case files
Manager	All branches, all cases	Assign work, invite staff, issue invoices, approve payment schedules, own any chapter, edit packages and the university catalogue
Coordinator	All cases and branches	Assign and route work, own any chapter; the front-desk role
Customer Service	Cases at their branch, leads, helpdesk	Route work, first-line support
Consultant	Their own caseload	Own chapters (consultations, applications, visa, departure), comment, request documents, update stages, raise draft invoices
Finance	Invoices, ledger, payments, packages	Issue, void and credit invoices, edit packages, approve payment schedules

Branch scoping is real: a role without "see every branch" is confined to its own branch, enforced by the API.

PART 4

The client journey

Everything hangs off one ladder: **six chapters**.

CHAPTER	WHAT HAPPENS
1: Consultation	Client books online or in-person, picks a real slot, pays, meets the consultant, gets an eligibility assessment
2: Enrolment	Client confirms they want to proceed (or pauses or declines), picks a service package and payment plan, pays the 10% agency deposit
3: Applications	Client chooses schools, pays the application fee, staff submit and track offers, client accepts one
4: Visa	Client consents to start, pays the visa fee, the officer files and tracks it to a decision
5: Departure	Travel help, the pre-departure fee milestone, a checklist, then official documents released
6: Complete	Client departed; post-arrival instalments run as aftercare

Inside the chapters the portal walks the client through about fourteen finer steps. A step is never silently skipped: if later evidence exists (an admission, a paid visa), earlier un-ticked steps show as "skipped", not done.

HOW "WHERE THE CLIENT IS" IS DECIDED

Two representations, deliberately different:

- **A coarse stage stored on the case**, set by staff and checked against gate rules (you can't enter visa work before the fees settle). It acts as a floor: it can push a client forward, never backward.
- **A fine step computed from facts**. The API gathers what actually exists (a booking, a paid deposit, an offer, a consent) and derives the step the portal shows. It is never stored, so it can't disagree with reality, and both apps use the same shared logic and the same words.

Everything is scoped to the client's **current case**. A returning client's earlier case can never leak paid invoices or admissions into the new one; schools and invoices are read per case, never per person.

CONSENT IS A GATE, NOT A SUGGESTION

The client's Confirmed / On hold / Declined is recorded and gates stage entry. Staff cannot push a client past a consent they haven't given. The visa chapter in particular opens only on explicit consent.

WHO OWNS THE WORK

Ownership history is append-only. Who owns a case now is the open record; reassignment closes the old one, never overwrites. Each chapter has its own seat (consultant, visa officer, travel officer), and the assignment picker for each chapter only offers staff holding the matching ownership permission. A consultant can delegate a session to a colleague and reclaim it, and a daily duty-coordinator roster covers each branch.

HARD BOUNDARIES

Some moments stop the case until a manager acts. Paying the deposit raises a "pending handler assignment" handoff; that payment is the only trigger for the school-submission handler. Entering departure or the final payment stage hard-gates the same way until a manager resolves the handoff or defers it.

FLEXIBLE ENTRY AND EXIT

Service packages are priced per stage. A client can enter at different service stages, select only what they need, stop after any reached stage, and later request to continue. Staff approve the continuation.

WHO DOES WHAT, WHEN

MOMENT	ACTOR	ACTION	CLIENT SEES
Booking lands	Coordinator or manager	Assign a consultant (or the duty roster covers it)	"Consultant being assigned"
Session held	Consultant	Complete the assessment (eligible / conditional / not eligible, with a note)	Assessment outcome
Enrolment open	Client	Confirm, hold or decline; pick package and plan; pay the deposit	Journey advances
Deposit settled	Manager	Resolve the handler handoff, assign the case consultant	Named consultant
Schools chosen	Chapter owner, then finance	Raise the draft invoice, approve and issue it	"Fee being prepared" then a payable invoice
Offers in	Consultant	Record each school's outcome	Live tracking; client accepts an offer
Visa start	Client, visa officer, finance	Client consents; officer raises the draft, finance issues, client pays	"Pay the visa fee"
Visa work	Visa officer	Embassy filing, biometrics, decision	Visa tracking
Departure boundary	Manager	Resolve the handoff, assign the travel officer	Departure chapter opens

Travel settled	Client and finance	Flight choice; fee milestone paid	Documents on the unlock path
Pre-departure	Client and staff	Checklist items verified or waived	Official documents released
Any time	Finance	Issue, void, credit invoices; edit packages	Ledger and receipts
Any time	Admin	Staff invites, roles, sign-in policy, content	(No client-visible step)

Consultations, scheduling and meetings

What a client can book comes from three inputs: the branch's weekly hours and slot settings, each consultant's own working hours, and everything already booked, including busy times mirrored in from staff personal calendars. A database-level rule makes double-booking impossible even if two people click at the same instant.

Bookings have a full lifecycle (cancel, reschedule, reassign, confirm, complete, no-show, rebook credit), and every transition is written to an append-only history. Reschedules a client requests go to staff for approval; staff-side reschedules are direct.

Online sessions run in video rooms the client joins **inside the portal**, no external link needed. A join window is enforced (opens a few minutes early, closes after). A background check every minute marks meetings live or ended, feeding the console's "what's happening now" view.

Staff calendars connect both ways: they can pull their personal calendar's busy times in so clients can't book over them, and publish their Century bookings out to their personal calendar as a subscribeable feed.

Money

All amounts are stored as integer cents. The invoice pipeline is a two-step approval chain that drives the journey:

1. The **chapter owner** (the consultant or visa officer doing the work) raises a draft estimate, a proforma, on the case.
2. **Finance** approves and issues it. Now it's a real, payable invoice.
3. The client pays; partial payments count; when the balance reaches zero the invoice settles.
4. Corrections happen by voiding or crediting. Every action lands in the invoice's permanent event history, never by editing the past.

HOW CLIENTS PAY

Paystack is the primary client-side rail: checkout, then verification, then a signed webhook settles the payment on the server. The webhook is the authority; if it's slow, staff can reconcile against the gateway. A Stripe card gateway exists alongside it and activates when a key is configured, so the payments log shows records from both gateways plus staff-recorded bank and cash entries. There is deliberately no way for a client to claim "I paid"; cash and bank payments are recorded by staff. Mobile-money charges with an OTP step are also supported. Receipts and invoices print as PDFs.

INSTALMENTS

Clients can pay in instalments: full payment or a plan at enrolment, and a post-arrival schedule they propose and a manager or finance approves. With the client's consent, a saved payment authorization lets the system charge each instalment automatically; every attempt is logged so failures are visible, never silent.

FEES AND PACKAGES

The fee catalogue defines what each charge is (agency service fees, plus third-party fees like embassy and school application charges), with tariffs per destination. Service packages are priced per stage so clients buy only the chapters they need.

THE LEDGER

Every case and every client has a ledger view: a chapter-numbered journal of invoices and payments. Underneath sits the invoice event history, immutable and append-only, the substrate the views are built from. A paid invoice can't be voided (it must be credited), and settlement is idempotent, so a replayed webhook or retried verification can never charge twice.

Documents

Clients upload through a vault: the file goes straight to private storage via a temporary signed link, then staff verify it or reject it with a note. Verification progress feeds the client's checklist and the journey on both sides. Staff can request a specific document by name ("passport data page"), which appears as a task in the portal. Rejected files are purged after a retention period. Nothing is publicly reachable; every download goes through a short-lived signed link the API mints.

Communication

ONE CONVERSATION SYSTEM, THREE SURFACES

A single thread model (conversations, members, messages, attachments, mentions, reactions) serves:

- **The helpdesk:** client-to-staff threads, scoped to a case and chapter, with subject, category, priority, who it's waiting on, first-response and resolution times, and a satisfaction rating when it closes.
- **The staff Communication Hub:** direct messages, group chats, mentions, reactions, attachments, presence, typing indicators.
- **The client's Communication Center:** the portal's thread with their assigned staff plus the AI assistant.

Threads can be escalated to managers, and staff keep a library of canned replies. When a client is offline, a reply is emailed to them, and their email reply lands back on the same conversation. Staff presence and typing are shown live as they happen.

THE AI ASSISTANT

A knowledge assistant runs at the edge on Cloudflare's Workers AI (a Llama 3.1 model, streaming its replies as they're generated), with three personas: a **website assistant** for prospective students on public pages, a **portal assistant** inside the client's communication hub, and an **enquiry** path for visitors. It answers general questions only, says so when it isn't certain of a Century-specific detail, and points people to a human for anything account-specific. First-time public use is gated by a bot check so the widget can't be farmed.

NOTIFICATIONS

One notification service fans out to three channels (in-app, email, and browser push) and logs every delivery attempt so a failure can be retried. Clients only ever see client-appropriate event kinds; staff-only signals never reach the portal. Each user controls their own matrix of which events arrive on which channels. Email never sends inside a web request; it's queued, so a failed send can never roll back a booking.

The portal is push-driven: a live event stream tells it when stages, assignments, invoices or visa states change, so the client sees updates without refreshing. Polling exists only as a fallback for a dropped stream.

THE AUDIT TRAIL

Administrative actions land in an append-only audit trail: who, what, on which record, from which address. Every entry seals the previous entry's fingerprint, so altering or deleting any row breaks the chain and a verification check proves it. The trail is browsable by day, filterable, exportable, and alert rules can watch it (for example, a burst of failed sign-ins or a role change). Settings changes have their own parallel trail.

Marketing

Production email marketing built on the suite's own data, in five console tabs: Campaigns, Audiences, Contacts, Templates, Automations.

PEOPLE, NOT ADDRESS BOOKS

The **person** is the record; a list is a membership. The contacts view gathers everyone the suite knows (mailing-list subscribers, applicants, leads), each with their real identity (an applicant shows their case and chapter, a lead their stage), their consent state, and their memberships. One person with two memberships appears once, never as two rows.

CONSENT IS EARNED, NEVER FABRICATED

Adding a contact has exactly two doors: send them a confirmation email (they become opted-in when they click), or record that they consented offline, which requires a written note as the audit trail. There is no silent "confirmed". CSV imports run a dry run first (how many are new, how many duplicates merged, how many invalid), then apply the same consent decision to the batch.

SUPPRESSION

One global do-not-contact list, checked twice: when a campaign's audience is frozen (only opted-in, unsuppressed people become recipients; everyone else is recorded as skipped with the reason) and again at the instant each email is sent. Someone who unsubscribes between scheduling and sending is skipped, not mailed. Bounces and spam complaints from the email provider suppress automatically.

SEGMENTS: LIVE AUDIENCES

Saved filters over the suite's real data (applicants by chapter, branch, country, offer state, unpaid milestone, departure window; leads by stage, source, no-show), evaluated at send time, never copied. A preview shows how many match, how many are opted in, never asked, or suppressed. A campaign targets a list or a segment, never both.

CAMPAIGNS

Composed from blocks (headings, paragraphs, buttons, dividers, two columns) with merge fields that pull real data: the person's name, case reference, stage, officer, branch, next due date, portal link. Senders get a preheader, a from-name, and a reply-to (the branch mailbox by default). Preview is the actual server-rendered email, and a test send goes through the real pipeline. Reports show delivered, open, click, bounce and unsubscribe rates, an hourly timeline, top links, and the per-recipient ledger with retry for failures. Every footer carries working unsubscribe and preferences links.

TEMPLATES AND AUTOMATIONS

Templates render as live miniatures; presets are read-only starters you fork. Automations are rules (when this happens, to people matching this segment, send this template after this delay) covering moments like a no-show follow-up, an assessment completed but no enrolment, an offer received, a visa approval, thirty days to departure, or an overdue milestone. Starters ship as drafts; staff turn them on deliberately. Every firing is logged, including why a person was skipped.

Content management

One identity record, one content store, one media library, read by the website, the portal, the console, and the email layout.

- **Brand:** the identity record (names, logos, colours, contacts, socials) has a draft and a published copy. Edits save to the draft; publishing makes it live and records who published, with full version history and revert.
- **Entries:** content collections keyed by name and slug, moving through draft, review, then published; schedulable and fully versioned.
- **Navigation:** the site's header and footer link lists.
- **Copy:** every label that's content rather than code, per surface (site, portal, console, email).
- **Media:** the image library. Uploads go straight to private storage, alt text and focal points are editable, and public reads pass through a short-lived signed link so nothing is openly reachable.

Public content reads are edge-cached: fast for visitors, fresh within a minute of publishing.

The Operations Center, page by page

Every view keeps its state in the address bar (filters, tabs, open records), so links survive refresh and notifications deep-link to the exact task.

- **Workspace:** the day's triage. Unassigned consultations, cases awaiting a handler, invoices to raise, documents to review, reschedule requests, banded overdue / today / later, with inline assignment. A caseload view shows who is carrying what across the team.
- **Inbox, Dashboard, Now:** what happened, the numbers, what's live.
- **Helpdesk:** the client conversation queue with its full lifecycle.
- **Chat:** staff messaging.
- **Consultations:** booked sessions. Assign, refer to another branch, delegate and reclaim, assess, reschedule approvals, no-show and credit, comments, document requests, and the full activity history.
- **Cases:** every engagement, list or board, with the full case file: stages, chapter owners, package and plan, invoices, comments, documents, visa and departure details, release override, post-arrival schedule, and referral to another branch.
- **Clients, Leads:** records and the enquiry pipeline.
- **Appointments, Live meetings:** the week and rooms in progress.
- **Universities, Programmes, Scholarships, Packages, Departure checklist:** the catalogue editors. Scholarships can also be awarded to a specific applicant from their record.
- **Invoices, Ledger, Payments, Fee schedule, Payment plans:** the finance suite.
- **Reports:** revenue and operations analytics.
- **Scheduling, My availability:** the week the branch offers, and each person's own hours and calendar connections.
- **Marketing:** campaigns, audiences, contacts, templates, automations.
- **System, Staff & roles, Clients directory, Authentication, Audit, Content (CMS), Lookups, Notifications, Settings:** platform administration. Health, the permission matrix, sign-in policy, the audit feed, content, form dropdowns, notification templates and integrations.

The website and portal, page by page

THE PUBLIC WEBSITE

Open to everyone, no sign-in:

- **Marketing pages:** home, about, why choose us, services, visa services, student services, success stories ("Red Seat"), FAQs, blog, events, contact.
- **The catalogue:** destinations, universities, programmes and scholarships, each browsable down to a detail page and served live from the same database the staff edit.
- **The enquiry widget:** a floating door on every page where a visitor leaves their details and becomes a lead in the CRM.
- **The newsletter:** a subscribe prompt, double opt-in confirmation, and unsubscribe/preferences pages.
- **"Start your journey":** the doorway that turns a visitor into a registered client and drops them into chapter one.
- **The AI assistant and a newsletter popup** float on every public page; first-time assistant use passes a bot check.

THE PORTAL

The sidebar is the journey itself, the six chapters as the spine, showing done, current and locked.

- **Home / Journey:** the spine, the current step, the next unlock.
- **Consultation:** choose online or in-person, pick a live slot, pay, join the call in-app, reschedule or cancel within policy.
- **Enrolment:** the assessment outcome, the decision, package and plan, the deposit.
- **Applications:** school choices, the fee invoice, per-school tracking, accepting an offer.
- **Visa:** consent, the invoice, live tracking to the decision (a refusal parks the case with a reapplication path).
- **Departure:** travel choice, the fee milestone, the checklist, document release.
- **Complete:** the post-arrival instalment schedule.
- **Appointments, Documents, Fees, Security:** bookings, the vault, the ledger and receipts, and the security page: profile fields, a two-step verified email change, MFA setup, session list, and avatar upload with crop.
- **The Communication Center:** a floating support channel inside the portal, the staff thread plus the AI assistant.
- **Newsletter pages:** confirm, unsubscribe, and preferences.

Data, security and the background engine

DATA POSTURE

Every table is behind row-level security. The app connects as owner, but any future non-owner path is denied by default rather than silently open. Anything that's evidence is append-only: audit entries, invoice events, booking events, assignment history, activity feeds. Corrections append; history is never edited.

Around the edges: requests are rate-limited and carry strict security headers, the origin allowlist is exact-match, public forms pass a bot check, sensitive staff actions can demand a fresh sign-in, and every account (client or staff) can list and revoke its own sessions.

DELETION, EXPIRY AND RETENTION

Different kinds of data leave the system in different, deliberate ways.

Removing a client account is a super-admin action, always audited, with three modes:

- **Disconnect** removes the login only. The applicant record, the cases and the files stay; the person simply can't sign in anymore.
- **Archive** marks the applicant record archived and removes the login. The history is kept but the person is closed out.
- **Purge** is the full removal: the person's conversations, comments, activity events and leads go, the applicant record and its cases are deleted, and every file they uploaded is cleaned from storage best-effort afterwards. One deliberate exception: **invoices are never deleted**. Journey invoices belonging to the purged case are detached and re-labelled as one-off charges, because financial records must outlive the case they came from.

Everyday removal follows the same spirit. Staff can remove leads, documents, canned replies, roles, packages, and marketing entities (draft campaigns, lists, templates, segments, individual suppressions, list members). Clients can withdraw a reschedule request, cancel autopay consent, remove a school choice, disconnect a calendar feed, or unsubscribe a device from push. Chat messages are never truly deleted: they're marked deleted and shown as a tombstone, because replies and forwards quoting them must not dangle. Open invitations can be revoked.

Things that expire on their own: staff invitations, one-time codes and verification links, sessions, signed file links (minutes for an upload, under an hour for a read), temporary consultation delegations, and a meeting's join window.

Things kept on a clock: rejected documents are purged by a daily sweep after a retention window; finished queue jobs are kept only briefly. **Things kept forever:** the audit trail, invoice events, booking events, assignment history and activity feeds are append-only and are never purged by anything.

BACKGROUND JOBS

Work that shouldn't block a request runs on a separate worker process: emails, campaign sends, push notifications, instalment charges, offline chat replies, helpdesk SLA checks, automation date-triggers, meeting status polls, calendar feed syncs, and document cleanup. Until that process runs, jobs simply wait. Bookings still succeed, but no email sends. It is not optional in production.

EXTERNAL SERVICES

Paystack for all payments, Resend for all email (delivery, open, bounce and complaint events flow back through a signed webhook), LiveKit for video rooms (with Daily kept as an alternative provider), Supabase for private file storage, Google for client sign-in and staff calendars, web push for browser notifications, and a pluggable text provider for phone sign-in (unconfigured by default; the feature refuses rather than pretending).

CONFIGURATION AND DEPLOYMENT

Production refuses to boot without its database, queue, signing secret and the browser-facing addresses. Misconfiguration fails loudly at startup, not silently at runtime. The API ships as a Docker image; the front ends deploy as Cloudflare Workers. The database is migrated by numbered files; new databases are built directly from the schema because the historical migration chain doesn't replay cleanly from zero. The rules and the reasons are written down in the migration discipline notes.

Under the hood, the flows that matter

Features above; mechanics here. These are the paths worth understanding before changing anything.

EVERY REQUEST, IN ORDER

A call reaches the API, gets security headers, passes the origin check, passes rate limiting, then meets the guard chain in a fixed order: prove you're signed in, prove you're staff (if the route needs it), prove you've enrolled a second factor (staff), prove your role may see the module, prove your role may do the action. A branch-scoped role gets its branch constraint applied last. The permission check reads the live roles table, so a role edit takes effect without a deploy; the built-in matrix is only a fallback. Inputs are validated by the same schema that generates the public API reference, so docs and enforcement can't disagree.

HOW THE JOURNEY STEP IS DECIDED

Nothing stores "step 7 of 14". Each time the portal asks, the API collects the facts about the client's newest case: does a consultation exist and what was its outcome, what did the client consent to, which package and plan, which invoices are paid, which handler is assigned, which schools are locked, which offers arrived, what the visa record says, what travel help was chosen, what checklist items are done. A single shared function turns those facts into the step list. Because it's computed fresh, the two apps can never disagree, and a fact that appears out of order (a visa paid before schools locked) marks the earlier steps "skipped" rather than pretending they were done.

HOW A BOOKING BECOMES A CONSULTATION

The client picks a slot, the API creates a payment, Paystack collects the money, and only then do the records land: the booking, the consultation, the invoice marked paid, the event history, and queued notifications. The double-booking guard is a database rule, not application hope: the column ranges physically cannot overlap, so two simultaneous checkouts can't both win the slot. Everything that can fail after the commit (the email, the meeting link, the push) is queued, so a provider outage can never roll back a paid booking.

HOW A PAYMENT SETTLES, TWICE, SAFELY

Two paths can settle a payment. The fast path is the client returning from checkout and the portal asking the API to verify. The authority path is the signed webhook arriving from Paystack on its own schedule. Both run the same settlement, keyed on the gateway reference: the first one to arrive writes the payment, the ledger entry, the event history, and the journey side effects (a deposit fires the handler handoff exactly once). The second arrival sees the reference already recorded and becomes a no-op. Retries, replays and double submissions cannot charge or advance twice. If neither lands, staff reconcile against the gateway to close the gap.

HOW INVOICES ARE NUMBERED AND TRACKED

Drafts (proformas) and issued invoices draw from separate number sequences, each allocated inside the same transaction as the insert, so there are no duplicate numbers under concurrency and no gaps from abandoned drafts. Paid totals are never stored as a number that can drift; they're summed from the payment records each time, and the next uncovered instalment due date is derived from how much has been covered. The ledger views are projections over the invoice event history, which is why they can't disagree with the payments table.

HOW A DOCUMENT MOVES

Upload is three steps: the API mints a short-lived upload link, the file goes straight from the browser to private storage, then the client tells the API it's complete and the record lands. Review changes the record's state (verified, or rejected with a note the client sees) and that state feeds the checklist and the journey. Downloads are minted per request. Rejected files are deleted by a daily sweep after a retention window, so rejected personal documents don't accumulate in storage.

HOW MARKETING ACTUALLY SENDS

When a campaign is sent, the audience is frozen into a recipient ledger: opted-in and unsuppressed people become pending rows, everyone else becomes a skipped row with its reason recorded. The worker then walks pending rows only, which is why a retry after a crash can never re-mail someone already delivered. Per recipient it builds the merge context from the real case (officer name, branch, next due), wraps every link for click tracking, sends through the provider, and stamps the outcome plus the provider's message id back on the row. As delivery events return through the signed webhook (open, bounce, complaint), the recipient row is updated and a bounce or complaint also writes the global suppression, so the address is protected everywhere from then on. The report is a read of the ledger, so numbers can never drift from what actually sent.

Automations work the opposite direction: a domain moment (visa approved, offer received, assessment complete, no-show, milestone overdue) fires an event into an intake check, which writes a scheduled send per matching person. A sweep every minute delivers whatever's due; a daily scan inside the same sweep handles date triggers like thirty-days-to-departure. Every firing is logged, including the skip reason.

HOW CONSENT FOLLOWS AN ADDRESS

Consent lives on the address, not on a list row. A confirmation click, an offline-consent note, a re-opt-in on the preferences page, an unsubscribe, a bounce: all of it lands in one consent ledger keyed by address. Any surface that asks "may we email this person" reads that ledger plus the global suppression list, so consent granted in the portal and a suppression caused by a bounce protect every list and every campaign at once.

HOW STAFF PERMISSION RESOLVES

A staff sign-in produces the same session as a client sign-in; what makes it staff is the staff record attached to the account, carrying role and branch. When a guarded route runs, the role's permission list is read from the roles table (a custom role's list is exactly what the permission matrix saved), falling back to the built-in defaults only when the role has no row. Seeing and doing are separate entries in that same list: "invoices" the module opens the page, "issue invoices" the capability enables the action. Ownership of a chapter is a capability too, which is why the visa-officer picker can only offer staff who may own visa work.

HOW CHAT STAYS LIVE

Each participant holds a read cursor on a conversation; unread counts are the distance between that cursor and the latest message. Typing indicators and staff presence ride on lightweight heartbeats rather than sockets. When a client is offline, a reply is emailed; when they answer the email, the reply is threaded back onto the same conversation by the token in the subject, so a thread started in the portal can continue in an inbox without forking.

THE AUDIT CHAIN, MECHANICALLY

Every audited write computes a fingerprint over the entry's content plus the previous entry's fingerprint, and stores both. Verification replays the whole chain: if any entry was edited or deleted, every fingerprint after it stops matching. It proves tampering happened, not just that data changed, which is why the console shows the chain status as a badge rather than trusting the rows.

THE SWEEPS AND THEIR CLOCKS

- Every minute: due automation sends are delivered; inside the same run, a daily pass handles date triggers.
- Every fifteen minutes: unclaimed helpdesk requests are auto-assigned so nothing sits unanswered.
- Every sixty seconds: meetings are marked live or ended.
- Daily: rejected documents are purged; date-based automation triggers evaluated.
- On a schedule: staff calendar feeds are mirrored into busy blocks.

Every queued job carries an idempotency key; a duplicate key is a silent no-op, failed jobs retry with growing backoff, and finished jobs are kept only briefly, so the queue can't fill with history.

HOW THE RECORDS RELATE

One account (`users`) may be a client, a staff member, or both shapes at once (a staff record attaches to the account). A client has a profile; each engagement is a case; a case owns school submissions, invoices, documents, comments, chapter seats and handoffs. An invoice owns lines, payments and events; payments point at gateway transactions; stored authorizations power instalment charges. A conversation owns members, messages, attachments and reactions. A campaign owns a recipient ledger and tracked links; the

consent ledger and suppression list stand beside all of it, keyed by address rather than by list. People in marketing are assembled at read time from three sources (list contacts, applicants, leads), which is why one person can never appear as two rows.

Covers the three deployables (public site + portal, Operations Center, API + worker) and the shared packages, as of the current codebase.